COMP717A: Internet of Things (2026)

IoT-Based Soil Moisture Monitoring and Irrigation Decision Support System

Group Members

Name	Student Number	Responsibilities	Contribution (%)
Varushka Dayal	223050219	User Authentication, Troubleshooting, Dashboard Modification, Documentation	20
Rahil Sha	223007960	Code Logic, Thresholds, Irrigation Decisions, Database Setup, Troubleshooting	20
Sasthi Naidoo	223004483	Connect/Calibrate Sensor, Raspberry Pi Implementation	20
Farha Mustan	223005933	Build Dashboard	20
Owethu Lushozi	223046815	Receive Sensor Readings, Backend/API Code, Connect Hardware to Software	20

Abstract

Efficient soil moisture monitoring is essential for optimizing water use, preventing plant stress, and promoting sustainable garden management. This project delivers a functional Internet of Things (IoT) prototype designed to capture soil hydration dynamics, log time-series data, and deliver actionable irrigation decision support through a secure web interface. Originally envisioned to manage multiple garden inputs under the COMP717A project statement, the system's physical scope was refined (with formal lecturer approval) to focus specifically on soil moisture telemetry, persistent storage, and interactive visualization due to budget and hardware constraints.

The physical architecture combines a Grove resistive soil moisture sensor with a DFRobot Gravity 16-bit analog-to-digital converter (ADC) and a Raspberry Pi 5 single-board computer. The Raspberry Pi communicates with the ADC over the Inter-Integrated Circuit (I^2C) bus to retrieve digitized voltage readings. Using Python and the smbus2 library, raw signals are mapped to relative moisture percentages based on empirical dry-soil (1,210) and water-saturated soil (13,990) calibration benchmarks. Readings are categorized into DRY (<30%), MODERATE (30%-69%), and WET (≥70%) operational states.

The backend software stack utilizes a local MySQL database for persistent time-series logging and user credential management. A responsive Flask web dashboard (secured via Flask-Login and Werkzeug password hashing) presents real-time moisture levels, historical trend line charts, and visual alerts when moisture falls below a critical 30% threshold. While automated actuator controls were excluded from the physical build, the prototype successfully demonstrates an end-to-end

telemetry pipeline, providing a reliable decision-support tool for home gardeners and a modular foundation for future automated irrigation expansion.

1. Introduction

Water scarcity and inefficient irrigation pose significant challenges to modern agricultural productivity and domestic garden management. Conventional watering practices frequently rely on visual observation or fixed calendars, which fail to reflect root-zone soil water dynamics and often lead to over-watering or crop stress. The integration of Internet of Things (IoT) technologies, combining field sensors, embedded processing units, cloud/relational storage, and web-based user interfaces, enables precise, real-time environmental monitoring. [2][3][6]

Recent research demonstrates that low-cost soil moisture sensors can effectively drive decision-support systems for precision irrigation. Studies evaluating low-cost capacitive and resistive probes highlight that while high-precision commercial sensors remain cost-prohibitive for many smallholders, lower-cost alternatives provide sufficient accuracy for threshold-based irrigation control when properly calibrated. Embedded single-board computers, such as the ARM-based Raspberry Pi, offer accessible platforms for local data collection, processing, and web server hosting. [1][4]

This project required us to develop an IoT-based garden monitoring system to minimize input waste and report garden health. Due to component and budget limitations, the implementation focused specifically on soil moisture acquisition, data persistence, and web-based decision support, receiving formal approval from the course lecturer for this adjusted scope.

This project addresses the challenge of accessible soil monitoring by designing a prototype IoT system that digitises analog soil moisture measurements, classifies soil hydric conditions, and presents actionable irrigation feedback via a secure web dashboard. The system replaces subjective visual estimation with structured data, helping users identify dry soil conditions and determine appropriate watering schedules.

The aim was to develop an IoT-based garden soil moisture monitoring and irrigation decision-support system using a Raspberry Pi 5, an external ADC, a resistive soil sensor, and a web dashboard.

The objectives:

- Integrate a Grove resistive soil-moisture sensor with a Raspberry Pi 5 using an external ADC.
- Establish communication between the Raspberry Pi and ADC, using the I²C protocol.
- Develop software to retrieve raw sensor readings and convert them into relative soil-moisture percentages.
- Implement moisture classification using DRY, MODERATE, and WET categories.
- Establish a database for storing relevant system data and sensor readings.
- Develop a web-based dashboard displaying the latest reading, moisture percentage, soil condition, historical measurements, and low-moisture alerts.
- Implement a threshold-based irrigation decision indicator using a 30% moisture threshold.
- Introduce login authentication to restrict access to the dashboard.
- Evaluate the implemented functionality and identify improvements required for a more comprehensive garden-management system.

The project scope encompasses hardware interfacing, signal conversion, data processing, relational database storage, and front-end visualization. The physical hardware consists of a Grove resistive soil moisture sensor, a DFRobot Gravity 16-bit

ADC, and a Raspberry Pi 5 running Raspberry Pi OS (64-bit). The software stack comprises Python 3, smbus2, MySQL, Flask, Flask-Login, and HTML/CSS/JavaScript.

Calibration was conducted using reference ADC readings in dry soil (1,210) and fully saturated soil (13,990) to map raw signals to a 0%–100% relative moisture scale. Moisture threshold boundaries were defined as: DRY: < 30%, MODERATE: 30% – 69%, and WET: ≥ 70%.

The system operates as a decision-support platform rather than a closed-loop automated controller. Due to budget constraints, physical actuators (water pumps, solenoid valves), flow sensors, automated probe power-switching circuits, and additional environmental sensors (pesticide/fertilizer dispensing monitors) were excluded from the hardware build. Sensor probe power is currently managed manually between monitoring sessions to mitigate electrode corrosion.

The rest of this report is as follows: System requirements and proposed solution, system design and implementation, Testing and Results, Experience and Challenges, Discussion, Limitations, and future work.

2. System Requirements and Proposed Solution

Functional Requirements

The functional requirements define the core operational capabilities of the IoT system across data acquisition, processing, storage, and web presentation:

Functional Requirement	Description
Analog Signal Acquisition	Retrieve analog voltage signals from the Grove resistive soil moisture sensor, convert them into 16-bit digital values via the DFRobot ADC, and transmit them to the Raspberry Pi 5 over the I²C bus
Signal Processing & Calibration	Compute relative soil moisture percentages (M) in Python using a linear scaling equation bounded by experimentally established dry (1210) and wet (13990)
Hydric Condition Classification	Categorise calculated moisture percentages into discrete operational states: DRY, MODERATE, and WET calibration references
Persistent Data Logging	Maintain a relational MySQL database to store time-series moisture logs, user account credentials, and system activity logs
Real-Time Data Visualization	Render current soil conditions, including the raw ADC reading, calculated moisture percentage, and categorized status indicator, on a web interface
Historical Trend Analysis	Plot time-series graphs on the dashboard to visualize soil moisture fluctuations over past monitoring sessions
Low-Moisture Alerting	Trigger visual dashboard alerts whenever soil moisture drops below the critical 30% threshold, prompting user attention

Irrigation Decision Support	Translate moisture measurements into actionable advice, informing users when manual watering is required
Access Control & Session Security	Restrict web dashboard access using Flask-Login session management and Werkzeug password-hash verification
Scope-Adjusted Garden Input Management (Partially)	Address soil hydration monitoring while deferring automated crop, fertilizer, pesticide, and pump controls due to hardware availability constraints, as approved by the course lecturer.

Non-Functional Requirements

The non-functional requirements define the attributes, safeguards, and constraints of the architecture.

Non-Functional Requirements	Description
Usability	Present complex sensor telemetry and moisture classifications through an accessible dashboard
Reliability	Ensure stable signal polling and reliable database operations across repetitive 5-minute sessions.
Performance	Maintain dashboard rendering and database query response times without noticeable latency during active user sessions.
Security	Enforce password-hashing techniques for stored credentials and apply API-key validation to protect the sensor ingestion endpoint against unauthorized submissions.
Maintainability	Construct the codebase using modular Python scripts, standard SQL schemas, and standard Flask templates to simplify troubleshooting and future updates.
Resource efficiency and mitigation	Limit active sensor energisation to intermittent monitoring windows to prevent accelerated galvanic corrosion of the exposed resistive probe electrodes.
Extensibility	Design the software architecture and GPIO pin allocations to seamlessly accommodate future actuators (e.g., solenoid valves, relay-driven pumps) and additional environmental sensors.
Cost effectiveness	Maximize functional capability using accessible, low-cost off-the-shelf components while detailing cost constraints for unequipped expansion modules.

Proposed IoT Solution

The proposed system uses an IoT-based approach to monitor soil moisture conditions and support informed irrigation decisions. Sensor measurements are acquired and processed by a Raspberry Pi 5 before being transmitted to a Flask backend hosted on a Windows laptop for storage in MySQL. An authenticated web dashboard presents current readings, historical trends, and

low-moisture alerts. Since the prototype does not include irrigation actuators, watering decisions and physical water application remain the responsibility of the user. The detailed system architecture is presented in Section 3.

3. System Design and Architecture

Overall System Architecture

The system consists of four principal layers: sensing hardware, processing, data storage, and presentation.

The sensing layer contains the Grove resistive soil-moisture sensor and the DFRobot ADC. The processing layer consists of a Raspberry Pi 5 and Python software that communicates with the ADC, processes measurements, and supports the application's backend. The storage layer uses MySQL to retain relevant system data and readings. The presentation layer consists of the web dashboard through which users view moisture information, historical trends, and alerts.

The data flow begins when the sensor responds to the conditions surrounding its probes. Its analogue output is digitised by the ADC and transmitted to the Raspberry Pi through I²C. The software processes the measurement, calculates a relative moisture percentage, and assigns a classification. The system stores relevant readings in the database and displays them through the dashboard.

The architecture also incorporates login authentication to restrict access to the dashboard. The system does not contain an irrigation actuator, and sensor power is switched off manually rather than through an electronically controlled switching circuit.

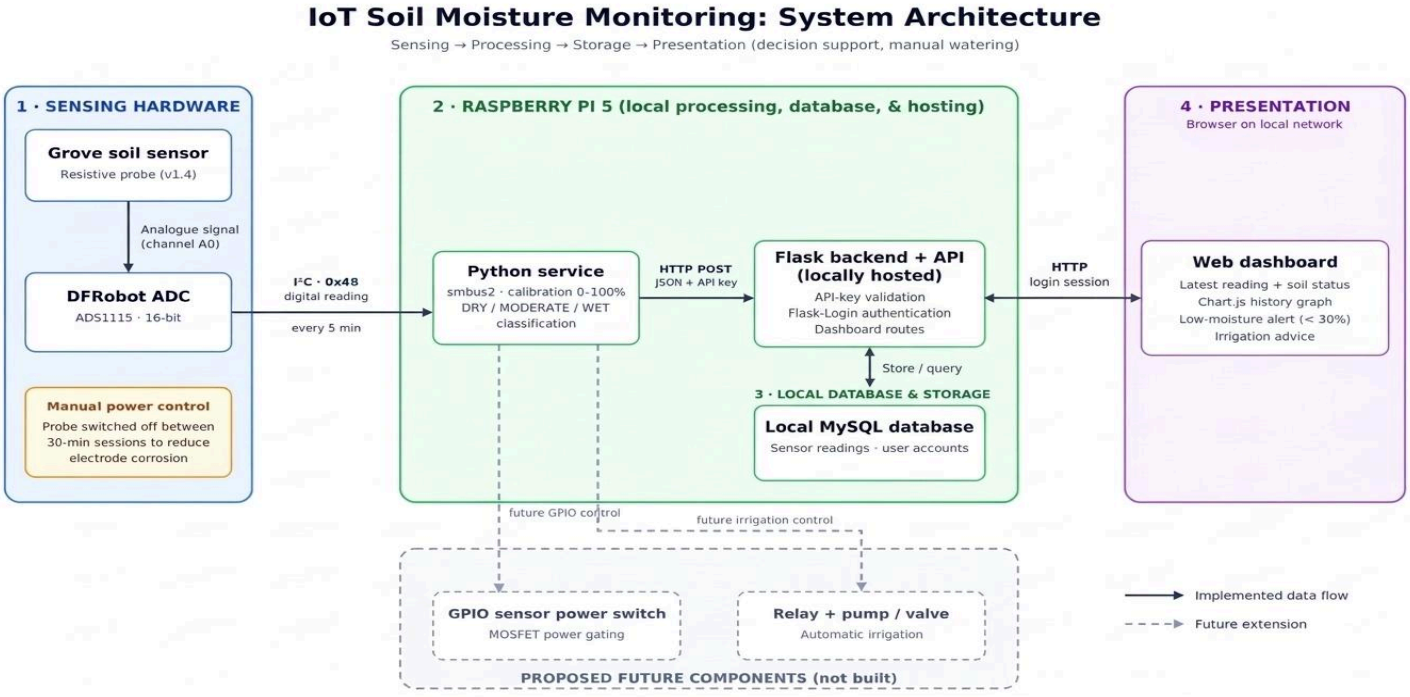


Figure 1: Overall system architecture. Architecture diagram illustrating the sensor, ADC, Raspberry Pi, Python processing, MySQL database, and web dashboard. Shows the sensor power-switching circuit and irrigation actuator as proposed future components rather than existing hardware

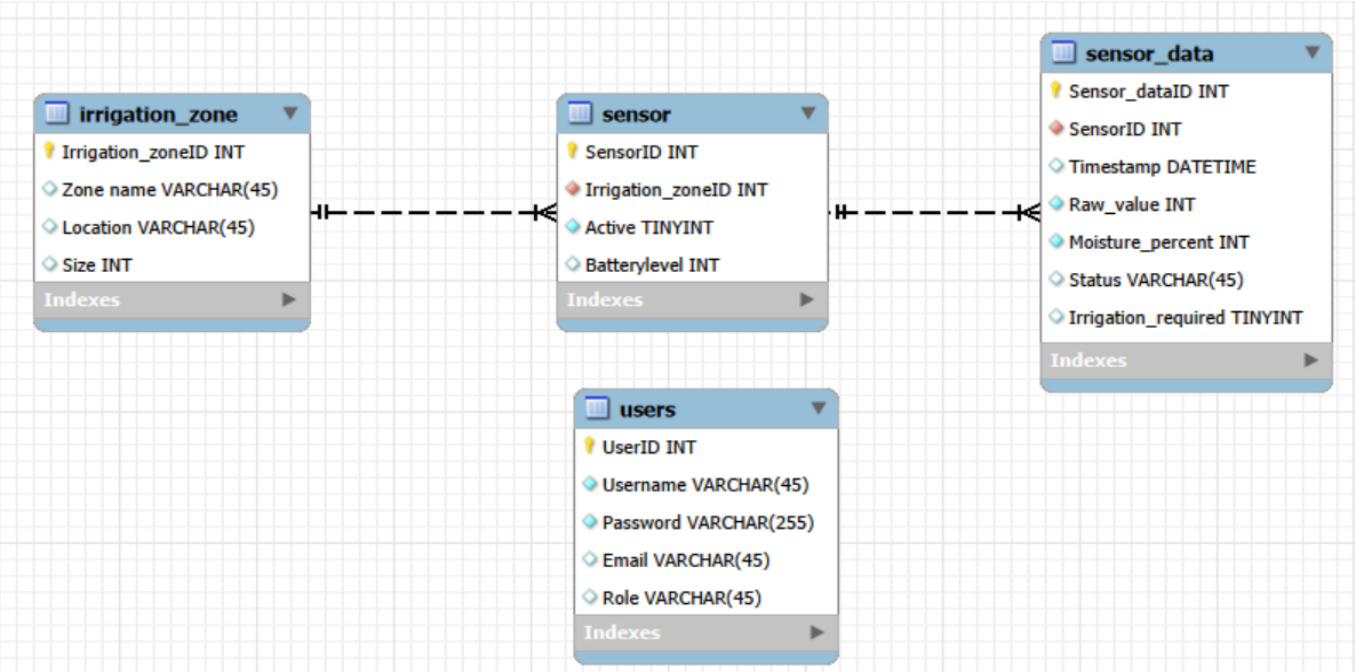


Figure 2: Database Design. ER diagram of MySQL tables for irrigation zones, sensors, readings, and users. A zone can have multiple sensors; each sensor can produce multiple timestamped readings (raw values, moisture %, soil status, irrigation recommendations). Users store dashboard authentication and access-control data.

Hardware Components and Connections

Raspberry Pi 5: Serves as the primary edge-processing device that is responsible for acquiring sensor measurements, processing soil moisture data, and transmitting the results to the backend server. An official 27 W USB-C power supply provides power to the Raspberry Pi and its connected components.

Grove Resistive Soil Moisture Sensor (version 1.4): Is used to detect changes in soil moisture conditions. The sensor operates through electrical resistance between its exposed conductive probes, which varies according to the surrounding soil conditions. It produces an analogue signal that is subsequently processed to estimate relative soil moisture.

DFRobot Gravity 16-bit ADC: It converts the analogue signal produced by the soil moisture sensor into a digital measurement that can be processed by the Raspberry Pi. The ADC communicates with the Raspberry Pi using the Inter-Integrated Circuit (I²C) protocol.

Hardware Connections: The soil moisture sensor is connected to the ADC's analogue input channel A0, while the ADC interfaces with the Raspberry Pi through its GPIO header using I²C communication. This arrangement enables the Raspberry Pi to obtain digital sensor measurements for subsequent processing and transmission to the backend system. The detailed wiring configuration and software-based sensor acquisition procedure are described in Section 4.

The system does not include a water pump, solenoid valve, or other irrigation actuator. Therefore, the implemented hardware supports soil moisture monitoring and irrigation recommendations rather than automatic water dispensing.

Software Architecture and Data Flow

Telemetry Processing Pipeline

The system processes soil moisture measurements through five stages:

1. **Data Acquisition:** The Raspberry Pi retrieves digital sensor readings from the ADC through the I²C interface.
2. **Moisture Processing:** Raw ADC readings are converted into relative soil moisture percentages using established calibration references.
3. **Condition Classification:** Moisture percentages are classified as DRY, MODERATE, or WET to support irrigation decisions.
4. **Data Transmission and Storage:** Processed readings are transmitted via Wi-Fi to the Flask REST API hosted on the Windows laptop and stored in MySQL.
5. **Data Presentation:** The web dashboard displays the latest measurements, historical trends, and irrigation recommendations.

The calibration equation, classification thresholds, and sensor acquisition procedures are detailed in Section 4.

Relational Database Schema

Data persistence is handled by a MySQL relational database (comp717_iot) hosted on the Windows laptop with two primary tables:

- **sensor_data**: Stores time-series data including Sensor_dataID (Primary Key, Auto-increment), Timestamp (DATETIME), Raw_value (INT), Moisture_percent (FLOAT), and Status (VARCHAR).
- **users:** Manages authentication records, including UserID (Primary Key), Username (VARCHAR, Unique), and Password (VARCHAR).

Web Application Architecture

- **Authentication**: Enforces session control using Flask-Login and Werkzeug. security password hashing (generate_password_hash / check_password_hash).
- **REST API Ingestion Endpoint:** Accepts POST requests carrying JSON-encoded sensor readings and validates submissions using a static API key header.
- **Dashboard Interface**: Serves dynamic HTML/CSS views rendered with Chart.js to plot time-series trend lines and highlight low-moisture alerts when readings cross below the 30% threshold.

Technology Selection and Justification

- **Raspberry Pi 5:** Selected for its Python support, GPIO connectivity, and ability to perform sensor acquisition and edge processing.
- **Grove resistive sensor:** Provided as a low-cost, accessible probe for detecting soil resistance variations
- **DFRobot ADS1115 ADC:** Selected to provide high-resolution analogue-to-digital conversion and I²C communication with the Raspberry Pi.
- **Python and smbus2:** Chosen for straightforward hardware communication, data parsing, and rapid integration with web and database libraries

- **MySQL:** Selected to ensure reliable, structured storage of time-series moisture records and secure user authentication tables. Also supports historical data retrieval.
- **Flask and Flask-Login:** Provide a lightweight Python web framework for serving dynamic dashboard pages, RESTful ingestion endpoints, and secure session management

4. System Implementation

Raspberry Pi Configuration

The Raspberry Pi 5 was configured as the primary edge-processing device responsible for acquiring and processing soil moisture sensor readings. Raspberry Pi OS (64-bit) was flashed onto a 64 GB USB 3.0 drive using Raspberry Pi Imager, as a microSD card was unavailable during implementation. Remote access and administration were established through Secure Shell (SSH) using PuTTY. WinSCP was used to transfer files between the Raspberry Pi and the Windows development laptop.

A dedicated Python virtual environment (venv) was instantiated to isolate dependencies. The Inter-Integrated Circuit (I^2C) hardware interface was enabled via raspi-config, and the smbus2 library was compiled to manage low-level register transactions. Device discovery was verified using i2cdetect -y 1, confirming stable communication with the external ADC at I^2C bus address 0x48.

Python-based scripts were developed to acquire raw ADC measurements, convert them into relative soil moisture percentages, and classify soil conditions as DRY, MODERATE, or WET. The processed readings are transmitted over the local Wi-Fi network to a Flask REST API hosted on the Windows laptop. The Flask backend handles the storage of received measurements in a central MySQL database, which provides the data used by the web-based monitoring dashboard.

Soil Moisture Sensor and ADC Integration

Since the Raspberry Pi 5 lacks native analog input channels, a DFRobot Gravity 16-bit analogue-to-digital converter (ADC), based on the ADS1115, was used to acquire measurements from the Grove resistive soil moisture sensor. The ADC was connected to the Raspberry Pi through the Inter-Integrated Circuit (I^2C) interface, with its power and ground terminals connected to the Pi's 3.3 V (physical pin 1) and ground (physical pin 6), respectively. The Serial Data (SDA) and Serial Clock (SCL) lines were connected to GPIO2 (physical pin 3) and GPIO3 (physical pin 5). The soil moisture sensor's signal, power and ground wires were connected to the ADC's A0 signal, positive and negative terminals, respectively.

Sensor acquisition was implemented in Python using smbus2 library to communicate with the ADC over I^2C. The monitoring script configures the ADS1115 registers and reads the conversion register to obtain the sensor's digital output. The two bytes returned by the ADC are combined and interpreted as a signed 16-bit reading. These raw measurements are subsequently used to calculate relative soil moisture percentages based on the established dry and wet calibration values. The sensor is sampled at five-minute intervals during the monitoring sessions.

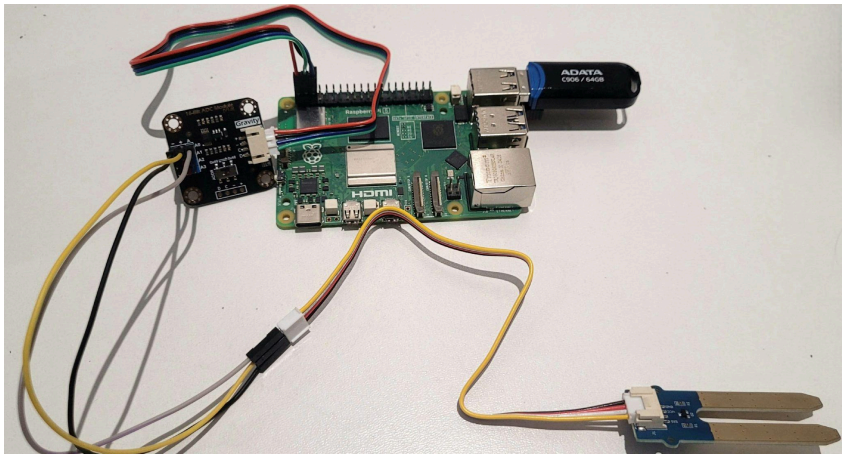


Figure 3: Physical hardware setup. Showing the Raspberry Pi 5, DFRobot 16-bit ADC, and Grove resistive soil moisture sensor.

Sensor Calibration and Moisture Classification

The soil moisture sensor was calibrated using readings obtained under relatively dry and very wet soil conditions. These measurements were used to establish reference values for converting raw ADC measurements into relative soil moisture percentages.

The calibration process produced a dry reference value of 1,210 and a wet reference value of 13,990. The moisture percentage was calculated using linear scaling between the two reference values, with the resulting percentage restricted to the range of 0% to 100%.

The calculated moisture percentage was classified into three predefined categories: DRY for values below 30%, MODERATE for values from 30% to below 70%, and WET for values of 70% or higher.

$M = max(0, min(100, \frac{R-1210}{13990-1210} \times 100))$. Where M is relative moisture percentage and R is the raw ADC reading.

These thresholds were used to provide a simplified indication of soil moisture conditions and to support the system's irrigation decision logic. The percentages represent values relative to the experimental calibration references rather than independently verified volumetric soil water content.

Sampling Strategy and Relational Data Logging

To protect the Grove resistive probe from accelerated galvanic corrosion caused by continuous DC energisation, monitoring is structured into 30-minute sessions with a 5-minute sampling interval. In this build, sensor power is cycled manually per session; implementing an automated MOSFET power-gating circuit remains a key target for future hardware iterations.

Data persistence is managed by a MySQL database. Processed readings are written to the sensor_readings table via mysql-connector-python, storing reading_id, timestamp, raw_adc_value, moisture_percentage, and status_category.

Web Dashboard and Telemetry Visualisation

A responsive, Flask-powered web dashboard serves as the primary user portal:

- **Live Status Widget:** Displays the latest calculated moisture percentage, raw ADC integer value, and categorized status badge (DRY, MODERATE, or WET).
- **Irrigation Decision Banner**: Provides automated watering guidance based on the 30% threshold indicator.
- **Historical Trend Graph:** Integrates Chart.js to render interactive time-series graphs from stored MySQL logs, helping users identify soil drying rates over time.

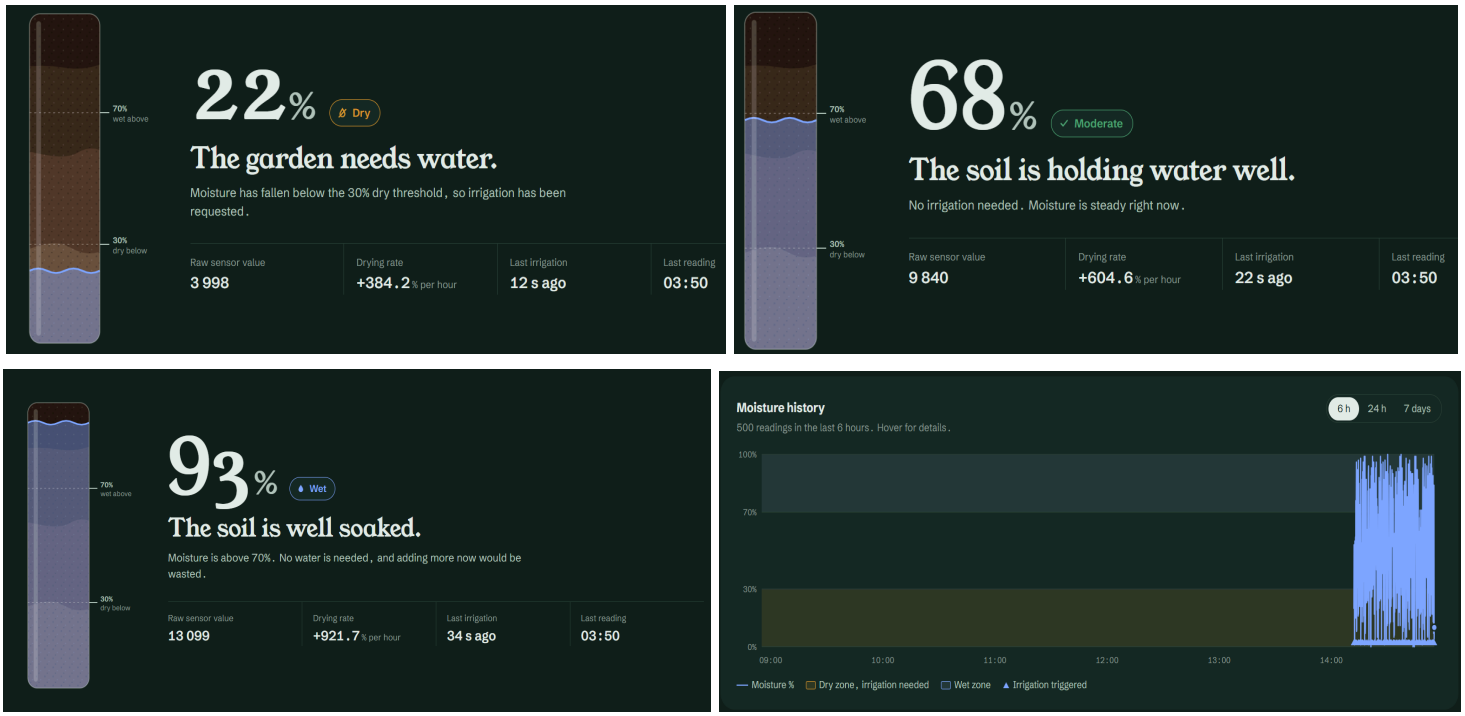


Figure 4: Web dashboard states and history view. Screenshots of the Garden Soil Monitor dashboard populated with simulated readings. (a) A DRY state (22%, raw ADC 3,998), where the headline reads “The garden needs water” and irrigation is requested. (b) A MODERATE state (68%, raw 9,840), where no irrigation is needed. (c) A WET state (93%, raw 13,099), where the soil is classed as well soaked. (d) The moisture history chart with the 6 h, 24 h and 7-day range selector, the shaded dry (below 30%) and wet (70% and above) zones, and irrigation-request markers. The soil core graphic on the left of each panel shows the current moisture level against the 30% and 70% thresholds. Readings were generated in simulation mode to exercise the interface, so the drying-rate and history values do not represent real soil behaviour.

Security and Access Control Implementation

Multi-layered security controls protect system resources and telemetry endpoints:

- Session Authentication: Dashboard routes are secured using Flask-Login. User accounts stored in the users table are authenticated by verifying submitted passwords against pbkdf2:sha256 hashes via Werkzeug's check_password_hash() function.

- API Key Protection: Remote sensor ingestion POST requests require a custom header matching a pre-shared API key (X-API-KEY), blocking unauthorized data injection attempts into the backend database.

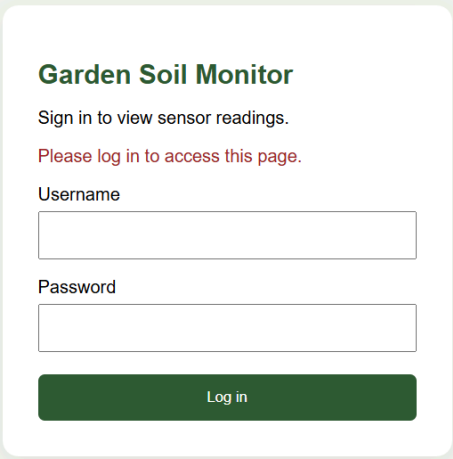


Figure 5: Dashboard login page. The Flask-Login sign-in page. Credentials are checked against password hashes stored in the Users table, and the dashboard and its data endpoints are only available after a successful login.

5. Testing and Results

Testing Methodology and Verification Framework

System testing evaluated hardware connectivity, sensor data acquisition, moisture calibration and classification, API communication, database storage, dashboard functionality, and user authentication. Evaluation combined physical sensor testing, controlled software test cases, and end-to-end monitoring sessions using real soil measurements.

Because reference gravimetric equipment and extended crop trials were unavailable during development, empirical validation focused on relative signal scaling, data pipeline integrity, and functional UI reactivity rather than absolute volumetric soil water content (VWC) precision.

Hardware Interfacing and Signal Acquisition Testing

I²C serial communication was verified on the Raspberry Pi 5 using i2cdetect -y 1. The DFRobot 16-bit ADC was successfully identified at address 0x48.

Sensor acquisition scripts using smbus2 consistently retrieved 16-bit conversion register values from channel A0. Physical testing confirmed that the raw ADC output responded dynamically to probe immersion depth and soil moisture variations:

Test	Procedure	Expected Result	Reported Outcome
I²C Bus Scan	Run I²C detect on Pi 5	Detect ADC at 0x48	Device acknowledged at address
Dry soil Exposure	Probe exposed to dry soil	Low raw ADC integer	Stable reading of 1210

Saturated soil	Probe submerged in saturated soil	High raw ADC integer	Stable reading of 13990
Data Ingestion API	POST payload with X-API-KEY	HTTP 201 Created and DB insert	Record written to sensor_data
Unauthorised POST	POST payload without API key	HTTP 401 Unauthorised	Request blocked. DB untouched

Moisture Calibration and Classification Validation

Using the empirical reference points, dry soil (1210) and saturated soil (13,990), the linear interpolation algorithm correctly mapped raw digital values across the full relative moisture spectrum (0%-100%).

Operational classification logic was verified across multiple hydration states:

- Dry Soil Conditions (Raw ADC = 2,488): Computed relative moisture was 10%, triggering the DRY classification badge and activating the low-moisture dashboard alert.
- Moderate Moisture Conditions (Raw ADC = 7,600): Computed relative moisture was 50%, yielding a MODERATE badge with no active alerts.
- Saturated Soil Conditions (Raw ADC = 11,434): Computed relative moisture was 80%, correctly triggering the WET badge.

System Integration: Storage, Dashboard, and Security

Data Persistence: The background ingestion service reliably parsed JSON payloads and logged records to MySQL. Historical queries executed cleanly without dropping records during 30-minute test sessions (5-minute polling interval).

Interactive Visualization: The Flask dashboard dynamically updated the live telemetry card and rendered historical trend lines via Chart.js. The 30% threshold indicator successfully highlighted drying trends.

Access Control: Unauthenticated requests to restricted routes (/dashboard) were intercepted and redirected to /login. Password verification correctly validated stored pbkdf2:sha256 Werkzeug hashes.

Evaluation Against Project Requirements

The prototype's operational capability was evaluated directly against the assignment criteria:

Requirement 1: Garden Monitoring & Targeted Input Delivery (Implemented): Real-time soil moisture tracking and a threshold-based decision-support system. Automated pesticide, fertilizer, and pump delivery were deferred due to budget limitations, with formal lecturer approval.

Requirement 2: Input Optimization and Resource Waste Elimination (Implemented): The 30% threshold indicator prevents unnecessary manual over-watering. While precise water savings could not be quantified without flow-metering hardware, the platform successfully replaces blind calendar watering with data-informed guidance.

Requirement 3: Garden State & Resource Usage Reporting (Implemented): The dashboard visualizes real-time and historical moisture logs, providing a clear record of soil hydration behavior over time.

6. Implementation Experience and Challenges

Challenges Encountered

Analog Signal Acquisition & ADC Integration

A primary hardware challenge was the Raspberry Pi 5's lack of native analog-to-digital converter (ADC) channels. Connecting the analog Grove resistive soil moisture sensor required integrating an external DFRobot Gravity 16-bit ADC (ADS1115) over the I²C bus.

Initial setup involved troubleshooting device discovery across Raspberry Pi OS (64-bit). Using i2cdetect -y 1, the ADC was successfully mapped to address 0x48. In Python, reading 16-bit values via the smbus2 library required bitwise shifting to concatenate two separate 8-bit register bytes into a single integer. Resolving endianness and register addressing nuances was essential for establishing stable signal ingestion.

Sensor Galvanic Corrosion & Power Management

Resistive soil moisture sensors pass a small DC current between exposed metallic electrodes. Continuous energization leads to rapid electrolysis and galvanic corrosion, causing probe degradation and severe measurement drift over time.

Ideally, sensor power should be dynamically switched via an N-channel MOSFET or PNP transistor controlled by a Raspberry Pi GPIO pin, energizing the probe only during active sampling windows. Due to component availability and budget constraints, this automated power-gating circuit could not be included in the physical build. To mitigate corrosion during testing, sensor power was manually disconnected between 30-minute monitoring sessions. This practical tradeoff protected hardware integrity while establishing a clear priority for future hardware revisions.

Empirical Sensor Calibration & Soil Texture Dynamics

Translating raw ADC count integers into meaningful relative soil moisture percentages (0%-100%) presented notable calibration challenges. Because resistive sensor readings vary with soil compaction, ambient temperature, and ion concentration, factory presets were inadequate.

To achieve reliable scaling, empirical bench trials were conducted to establish baseline boundary conditions:

- Dry soil Reference: 1210
- Water-Saturated Soil Reference: 13,990

A linear clamping function was implemented in Python to map raw values to percentage ranges. While this calibration proved effective for threshold-based decision support (DRY, MODERATE, WET), it highlights the inherent limitations of low-cost

resistive sensors compared to high-frequency capacitive probes, which measure dielectric permittivity and are less sensitive to soil salinity and electrode wear.

Scope Alignment & Resource Management

The original COMP717A project outlined a comprehensive smart garden framework encompassing automated irrigation pumps, pesticide/fertilizer delivery, and crop management. However, component sourcing delays and strict budget limits necessitated a pragmatic reduction in scope.

Through formal consultation with the course lecturer, the project focus was refined to deliver a robust end-to-end telemetry system, prioritizing accurate moisture sensing, database persistence, low-moisture alert logic, and secure web visualization. Navigating this scope adjustment provided valuable experience in agile project management and realistic engineering prioritization.

Full-Stack Software Integration & Access Control

Integrating Python edge polling scripts, a MySQL relational database, and a Flask web dashboard required careful software layering:

- **Database Concurrency:** Preventing database lockouts during simultaneous background sensor writes and user web queries required structuring clean database connection contexts in Python.
- **API Security:** To secure the ingestion route (/api/sensor-data), a pre-shared API key header (X-API-KEY) was implemented, preventing unauthorized POST submissions.
- **User Authentication:** Protecting dashboard access was achieved using Flask-Login and Werkzeug password hashing (pbkdf2:sha256), ensuring session integrity without adding unnecessary framework overhead.

Positive Experiences and Lessons Learned

Hands-On Hardware–Software Integration

Building an end-to-end IoT prototype provided invaluable practical experience in bridging physical hardware with high-level software stacks. Connecting a sensor through an analog-to-digital converter (ADC) and reading raw registers over I²C using Python demystified low-level embedded communications. Seeing real-time changes in soil moisture accurately reflected on a custom web dashboard was immensely rewarding and validated the full telemetry pipeline.

Technical Problem-Solving & Embedded Diagnostics

Working with Raspberry Pi 5 hardware offered deep insights into system setup, hardware pin mappings, and I²C troubleshooting using Linux utilities (i2cdetect). Learning to handle raw 16-bit register bytes via bitwise manipulation in smbus2 developed strong foundational skills in embedded debugging and signal processing.

Real-World Sensor Physics and Maintenance

Evaluating the Grove resistive probe highlighted crucial practical engineering realities:

- **Galvanic Corrosion:** Observing how continuous excitation currents degrade metallic electrodes underscored the importance of hardware longevity and automated power-gating strategies.
- **Empirical Calibration:** Discovering that factory sensor defaults rarely reflect real-world conditions taught the importance of empirical bench testing and calibration modeling across dry and saturated states.

Full-Stack Mastery & System Security

Developing the software infrastructure built proficiency in modular system design:

- **Relational Data Management:** Structuring clean MySQL schemas for time-series sensor data and user credentials ensured reliable data logging and fast query response times.
- **Web & Ingestion Security:** Implementing Flask-Login session authentication alongside API key validation (X-API-KEY) provided hands-on experience in securing web endpoints against unauthorized access and data injection.

Scope Management and Resilience

Adapting the project scope in response to hardware availability constraints was a major lesson in engineering adaptability. Collaborating with team members and consulting with the lecturer to pivot toward a robust, soil-moisture-focused decision-support platform demonstrated that a polished, fully functioning core system is far more valuable than an incomplete, over-scoped build.

7. Discussion

Efficacy of Threshold-Based Decision Support

Translating raw analog sensor signals into relative moisture percentages and categorised hydric states (DRY, MODERATE, WET) transforms unintelligible ADC integers into actionable intelligence. Rather than relying on rigid, calendar-based watering schedules or subjective visual inspection, the platform alerts users when soil moisture drops below the critical 30% threshold.

This decision-support approach directly aligns with frameworks highlighted by the World Economic Forum on digital water management, which emphasize that accessible, real-time telemetry can significantly optimize water allocation and mitigate plant hydration stress. While closed-loop automated irrigation (using pumps or solenoid valves) eliminates manual intervention, decision-support architectures provide a safer intermediate operational model when automated safety interlocks and flow-monitoring hardware are absent.[2][3][6]

Sensing Mechanics: Resistive vs. Capacitive Dynamics

The Grove resistive soil moisture sensor measures electrical resistance across two exposed metallic probes inserted into the soil matrix. While resistive probes are exceptionally low-cost and simple to interface, academic evaluations demonstrate that their readings are inherently sensitive to soil electrical conductivity (salinity), soil density variations, and temperature fluctuations.[1]

In contrast, high-frequency capacitive probes measure the soil's apparent dielectric permittivity, making them significantly more accurate and resilient against ionic interference. Nevertheless, studies by Adla et al. confirm that when properly calibrated using soil-specific benchmarks, low-cost resistive sensors offer acceptable accuracy for basic threshold classification, serving as a viable entry point for smallholder or domestic garden monitoring. [1]

Relative Calibration vs. Volumetric Precision

The two-point linear scaling function established in this project (bounded by dry-soil and water-saturated soil benchmarks) successfully normalizes sensor readings to an intuitive percentage scale. However, it is important to distinguish relative soil moisture from independently verified Volumetric Water Content.

Achieving true VWC accuracy requires rigorous thermo-gravimetric laboratory calibration curves tailored to specific soil textures (e.g., sand vs. clay-loam). For localized garden decision support, however, tracking relative drying trends and threshold boundaries provides sufficient operational value without requiring expensive soil-physics equipment[3][4].

8. Limitations

Despite successfully meeting its revised project objectives, the prototype exhibits several technical and hardware limitations:

- ### Lack of Physical Actuation & Water Metering:

  The system functions strictly as an information and decision-support tool. Because no water pump, solenoid valve, or relay module was incorporated, the physical application of water remains entirely manual. Furthermore, the lack of an inline flow meter prevents the system from measuring actual volumetric water consumption or quantifying exact water savings.

- ### Univariate Environmental Sensing:

  Due to budget and component sourcing limits, environmental parameters such as ambient air temperature, atmospheric humidity (e.g., via DHT22), ambient light, and soil nutrient/fertilizer concentrations were excluded. Consequently, the dashboard cannot factor evapotranspiration rates or weather forecasts into its irrigation recommendations[6]

- ### Single-Node Edge Architecture & Security Scale:

  Flask, MySQL and our website runs on a laptop. Ingestion API endpoints rely on pre-shared static API keys (X-API-KEY) rather than dynamic token-based authentication (JWT/OAuth2). The current setup lacks multi-tenant cloud hosting or long-range wireless networking (such as LoRaWAN) to support distributed multi-sensor field deployments[3]

9. Future Work

To evolve the current prototype from a decision-support tool into a fully automated, commercial-grade precision agriculture platform, future development should focus on five key areas:

1. Automated Sensor Power-Gating

Integrate an N-channel MOSFET or PNP transistor switch driven by a Raspberry Pi GPIO pin. Energizing the resistive probe only during active sampling windows will eliminate continuous excitation currents and prevent galvanic probe corrosion.

2. Closed-Loop Actuation & Water Metering

Incorporate 5V/12V relay modules connected to solenoid water valves or submersible pumps to execute automated watering. Adding an inline Hall-effect flow meter will enable precise volumetric water measurement and log exact water savings.

3. Multivariate Environmental Sensing

Expand the sensor array to include DHT22 ambient temperature/humidity sensors, solar irradiance meters, and soil NPK/salinity sensors. Integrating atmospheric data will allow the backend to calculate evapotranspiration (ET) rates and adjust watering schedules dynamically.

4. Capacitive & Digital Probe Upgrades

Replace low-cost resistive sensors with high-frequency capacitive probes or SDI-12 industrial sensors to eliminate salinity interference and improve long-term measurement stability.

5. Distributed Wireless Architecture & Cloud Scale

Transition from a single-node setup to a multi-node network using low-power ESP32 microcontrollers and LoRaWAN or MQTT protocols, enabling large-scale field deployments linked to a centralized cloud backend.

10. Conclusion

This project successfully designed, implemented, and evaluated a functional Internet of Things (IoT) soil moisture monitoring and decision-support prototype. Built on a Raspberry Pi 5 single-board computer, the system bridges analog physical sensing and web-based telemetry using a Grove resistive soil moisture sensor and an external DFRobot Gravity 16-bit ADC (ADS1115) communicating over the I^2C bus.

Through Python software and empirical bench calibration, raw digital counts are dynamically converted into relative soil moisture percentages (0%-100%) and mapped to three operational categories: DRY ($M<30\%$), MODERATE ($30\% \leq M<70\%$), and WET ($M \geq 70\%$). Data persistence is handled by a local MySQL database, while a responsive Flask web interface, secured

via Flask-Login and password hashing, renders real-time telemetry, historical trend lines, and automated low-moisture alerts when readings cross below the critical 30% threshold.

While hardware availability and budget limits required refining the original COMP717A assignment scope to defer automated actuators and multi-input dispensers, the resulting platform delivers an effective, data-driven alternative to subjective calendar watering. It fulfills the core objectives of modern precision agriculture by transforming raw environmental signals into actionable insights, providing a robust, modular foundation for future smart garden expansion.

Looking ahead, integrating automated MOSFET power-gating, capacitive sensors, and relay-driven pumps will enable a smooth transition from manual decision support to closed-loop automated irrigation. Ultimately, expanding the architecture with multivariate environmental sensing and a distributed LoRaWAN wireless network will establish a highly scalable, commercial-grade platform for modern smart agriculture.

AI Declaration

We made use generative AI in this report in the following areas:

- ChatGPT was used to assist with assembling hardware, troubleshooting, detecting inconsistencies in the report and code organisation.
- Co Pilot assisted with coding and debugging.
- Claude assisted in code generation and debugging.
- Grammarly helped with sentence structure in the report.

While generative AI was used, all work had been checked and edited, and all authors of this report claims responsibility for the work produced.

References

[1] S. Adla, N. K. Rai, S. H. Karumanchi, S. Tripathi, M. Disse, and S. Pande, “Laboratory Calibration and Performance Evaluation of Low-Cost Capacitive and Very Low-Cost Resistive Soil Moisture Sensors,” Sensors, vol. 20, no. 2, p. 363, Jan. 2020, doi: 10.3390/s20020363.

[2] P. Tan, G. E. Teshome, R. M. Sezadur, S. Hasan, and X. He, “Design and Implementation of Soil Moisture Monitoring and Irrigation System Based on ARM and IoT,” 7th International Conference on Intelligent, Interactive Systems and Applications, vol. 208, pp. 486–493, Jan. 2022, doi: 10.1016/j.procs.2022.10.067.

[3] P. Vandôme et al., “Making Technological Innovations Accessible to Agricultural Water Management: Design of a Low-Cost Wireless Sensor Network for Drip Irrigation Monitoring in Tunisia,” Smart Agricultural Technology, vol. 4, Art. no. 100227, Aug. 2023, doi: 10.1016/j.atech.2023.100227.

[4] J. D. González-Teruel, R. Torres-Sánchez, P. J. Blaya-Ros, A. B. Toledo-Moreo, M. Jiménez-Buendía, and F. Soto-Valles, “Design and Calibration of a Low-Cost SDI-12 Soil Moisture Sensor,” Sensors, vol. 19, no. 3, p. 491, Jan. 2019, doi: 10.3390/s19030491.

[5] W. H. Mahdi, B. N. Khalaf, O. A. Mahmood, and M. N. Ghazal, “Design and Implementation of an IoT System for Smart Irrigation,” Mathematical Modelling of Engineering Problems, vol. 12, no. 9, pp. 3325–3334, Sept. 2025, doi: 10.18280/mmep.120935.

[6] World Economic Forum, “Harnessing Digital Technologies for Smarter Water Management in Agriculture,” World Economic Forum, June 04, 2025. https://www.weforum.org/publications/harnessing-digital-technologies-for-smarter-water-management-in-agriculture (accessed Oct. 25, 2026).